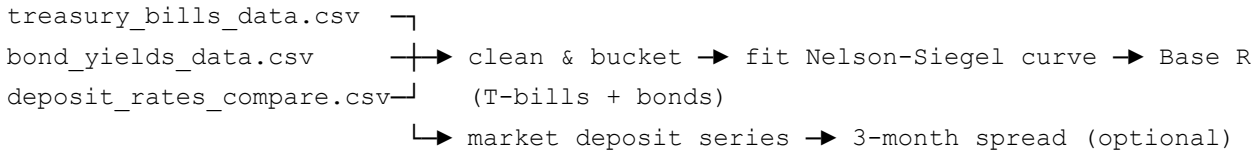
How We Build the Base Curve — Code & Data Walkthrough

Here's the base rate construction step exactly as our code runs it, walked through with real intermediate table shapes from an actual run against representative input files — not illustrative numbers we made up for the doc. All of this lives in `curve_model/ftp_curve_model.py`, orchestrated from `curve_model/01_curve_construction_demo.ipynb`.

The short version of how we get there:



We feed the base curve two independent inputs, and we never let them mix into a single fit. T-bills and bonds get fitted into a smooth curve shape on their own. The market deposit series only ever gets a supporting role — an optional correction number, a 3-month shift, off by default (see `curve_model/CURVE_CONSTRUCTION_ENGINE.md` for why). Even when we do enable it, deposits shift the fitted curve; they don't get fitted alongside it.

Step 1 — Raw data, as it actually arrives

We work with three files, three different shapes. `treasury_bills_data.csv` is one row per auction:

Local Code	Average Discount Rate	Average Competitive Yield	Settlement Date	Maturity Date	Number of Days
TB-0741	7.80%	7.96%	4-Jan	6-Apr	92
TB-0742	7.80%	7.95%	12-Jan	13-Apr	91

`bond_yields_data.csv` is the opposite shape — one row per **observation date**, one column

per **bond** (identified by its own maturity date), values already in percentage-point form (`13.427` means 13.427%, no `%` sign):

Maturity date	31-Jan-24	31-Oct-26	30-Jun-24
31-Aug-10	13.427	16.178	13.841
1-Sep-10	13.425	16.175	13.838

`deposit_rates_compare.csv` is small and wide — one row per tenor, one column per **competitor bank** (we don't quote our own rate into this model at all — every column here is a market/competitor observation):

tenor	comparator_1	comparator_2	comparator_3
0.08	3.15	2.35	2.65
0.16	—	—	—
0.96	6.75	—	3.30
4.82	7.45	—	3.45

Step 2 — Cleaning & parsing (`load_tbills` / `load_bonds` / `load_deposits`)

Whatever shape a source arrives in, each loader's job is the same: strip formatting (`%` signs, whitespace), convert to numeric, compute a tenor in years, and land on one common two-column shape — `Tenor`, `Yield` (decimal). T-bills reduce from several raw columns down to just what the curve actually needs:

Tenor	Yield
0.249144	0.0920
0.747433	0.1024

We resolve deposits to **one series**, not two: a primary competitor column, with up to two further competitor columns tried in priority order wherever the primary is missing at a given tenor — a genuine market rate, never our own book, feeds every step downstream (the optional 3-month shift, and the liquidity premium fit in `docs/liquidity_premium_construction.md`). Any remaining gaps get filled by

inter/extrapolating that resolved series' own points — and we log an explicit warning whenever a gap needs true extrapolation beyond the observed range, rather than interpolation within it.

Step 3 — Preparing the data (`DataObject.prepare_data`)

Two things happen before anything gets fitted: a monotonicity check, and bucketing onto standard tenor points.

Monotonicity check — we log a warning (we don't silently fix it) whenever yield decreases where it shouldn't as tenor increases.

Bucketing — every raw series gets mapped onto our standard tenor grid (overnight, 30d, 60d, 3M, 6M, 1Y, 2Y … 30Y) with a discount factor alongside. We also build exponentially-weighted windows (1-month, 6-month) next to the single-day "latest" snapshot, since a lone day's T-bill data is usually too thin on its own to fit reliably.

Step 4 — The short-end spread (`fit_short_end_spread`) — optional

This is the 3-month shift we mentioned above, in code form: we take T-bill and *market* deposit yields (bucketed, ≤1 year), and compute the spread at each short tenor. Only the **3-month value** gets used when the shift is switched on — our single anchor point, on the reasoning that it's typically the most liquid, most reliably observed short tenor. We keep it disabled by default in this repo (see `curve_model/CURVE_CONSTRUCTION_ENGINE.md` — Known limitations — for why an unshifted base curve leaves the downstream liquidity-premium fit more exposed to short-end sparsity).

Step 5 — Fitting the Nelson-Siegel base curve (`fit_base_curve`)

We concatenate T-bills (≤1 year) and bonds (1-10 years) into one input table. `build_ns_bounds_base` / `build_ns_init_base` / `build_ns_weights_base` turn that into bounds, a starting guess, and per-point weights; `fit_ns` then runs a constrained optimization with multiple random restarts, reporting whether they converge to the same best fit (a good signal of a stable solution) along with the resulting Nelson-Siegel parameters (β_0 level, β_1 slope, β_2 curvature, τ hump-decay) and fit diagnostics (SSE, RMSE).

Step 6 — Applying the (optional) shift (`apply_shifts`)

We combine the fitted curve (`base_term`) and the 3-month shift (`shift_3m` , zero when disabled) with one line: `base_term_shifted = base_term + shift_3m` (plus an optional scenario shift, zero in the base case). Same shift, added everywhere — a true parallel shift, never a re-fit.

`Base Term (shifted)` is what we call the **Base Rate** everywhere downstream — checked against our configured limits (`check_limits`), and the figure we add the liquidity premium to (`combine_full_curve`) — see `docs/liquidity_premium_construction.md` .

Quick function map

Step	Function	Input	Output
1-2	`load_tbills` , `load_bonds` , `load_deposits`	Raw CSVs	Clean `Tenor` / `Yield` tables
3	`DataObject.prepare_data`	Clean tables	Monotonic-checked, bucketed tables
4 (optional)	`CurveModelObject.fit_short_end_spread`	T-bill + market deposit buckets	`shift_3m`
5	`CurveModelObject.fit_base_curve`	T-bill + bond points	`ns_params_base` , `base_term`
6	`CurveModelObject.apply_shifts`	`base_term` (+ `shift_3m` if enabled)	`base_term_shifted` (**Base Rate**)